本章路線

這章要回答的核心問題是：多個執行流如何共享 process 資源，同時維持排程與資料正確性？

前置知識。 process 位址空間、CPU 排程基本概念、stack 與暫存器。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

學習目標

- 區分 process 與 thread 的資源邊界
- 比較 threads 映射模型與並行來源
- 辨識 cancellation、signal、TLS 與 fork 的執行緒問題

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.1 Overview

- thread 共享 process 的 code、data、heap 與開啟資源，但有各自的 program counter、暫存器與 stack。
- 建立或切換 thread 通常比 process 輕，但共享越廣，race condition 的影響也越大。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.1 Thread的私有執行context與共享process資源

Thread有自己的PC/registers、stack、執行/排程狀態及thread identity；同process通常共享code、globals/heap、address space與open-file資源。這種共享讓合作方便，也讓一個thread的bad pointer可能破壞其他thread的資料。不同stack是邏輯所有權與使用區段，不是同address space中的硬體防火牆；若把stack pointer傳給另一thread且物件仍活著，對方可能可存取它。

兩個workers的資料	W1	W2	正確協調
register/local index	自己的index0	自己的index0	獨立執行context
scratch buffer	B1暫存110	B2暫存120	各自持有，避免重疊寫
shared job queue	取job11	取job12	以同一同步協議避免重複取
global total	合併110	合併120	atomic/mutex或join後單方reduce

Responsiveness可來自UI thread繼續服務而worker等待I/O；economy來自共享資源而少做某些process建立成本；scalability依賴多核與可分工作。它們是可能效益，不是所有workload固定倍數。更多threads不會自動解除共享鎖、記憶體頻寬或序列依賴。

4.1 同一global不是兩份TLS

固定兩threads各increment一次counter。若counter是共享非atomic C/C++物件且未同步，衝突存取構成data race，語言行為未定義；load-add-store的讀0/讀0/寫1/寫1只是抽象機制反例，不是宣稱UB唯一輸出。

若改成每thread TLS counter，各自0→1，沒有同一counter上的共享寫；join後由main收集兩份並加總2。TLS可跨函式呼叫維持該thread的值，與某次函式的automatic local lifetime不同；由thread pool重用worker時，TLS不會每個job自動清零，需要job-boundary初始化/清理。

Thread結束前不能讓它的stack-local結果被回傳成失效pointer。資源共享與物件lifetime是不同條件；即使加mutex，也不能讀取已結束stack物件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.2 Multicore programming

- concurrency 表示任務在時間上重疊，parallelism 表示多個核心真的同時執行。單核可併發但不能平行。
- 平行化要拆分工作、協調共享資料並減少核心間同步。資料競爭與 false sharing 會抵銷加速。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.2 Concurrency、parallelism與資料/任務分解

Concurrency表示多條工作流可交錯前進，單CPU也能做到；parallelism表示同時執行，需足夠可用hardware execution resources。硬體SMT contexts共享core資源，不等於每個logical CPU都提供一份完整獨立core算力。

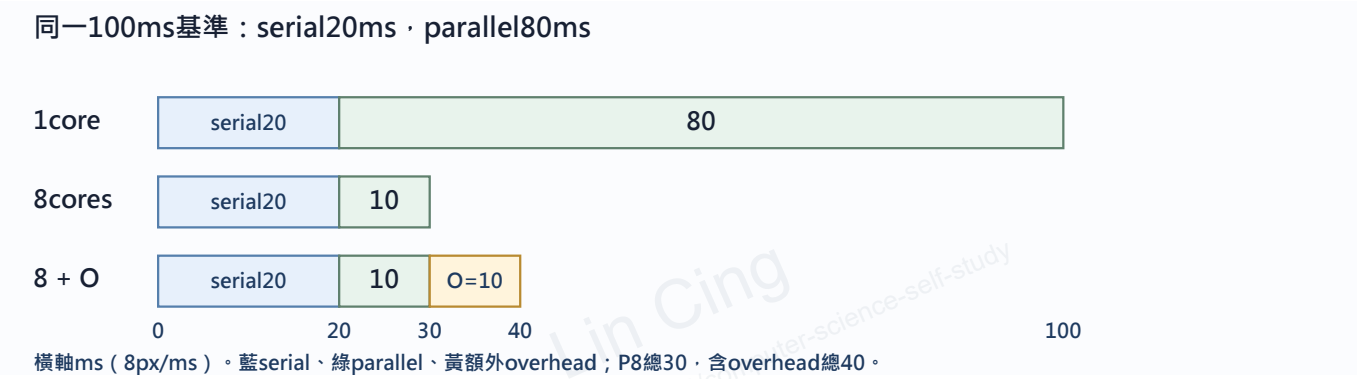
Data parallelism把同一操作套到不同資料partition；task parallelism讓不同功能/階段並行。分解需處理分割、負載平衡、相依、資料共享與測試。固定64MiB的32-bit integer array，共16,777,216 elements，分8份各8MiB/2,097,152 elements；每worker使用足以容納總和的64-bit accumulator先local sum，再合併8個partials只需7次加法。它減少共同counter更新，並未省掉讀完整陣列。

若每element都atomic-add global counter，會有16,777,216次共享RMW；partition版只在完成時發布8個partials。這是操作計數差異，不是已測得某個speedup。當前效能仍受bandwidth、cache與NUMA配置等影響。

4.2 Amdahl以基準時間比例建立上界

固定同一problem size，T1=100ms，其中20ms必序列，80ms可理想均分P個cores；無額外開銷。令s=0.2是基準時間比例，則TP=100(s+(1−s)/P)，speedup=1/(s+(1−s)/P)。P8時TP=20+10=30ms，speedup=10/3，efficiency=(10/3)/8=5/12；P→∞時speedup≤5。

4.2 Amdahl以基準時間比例建立上界 (續)

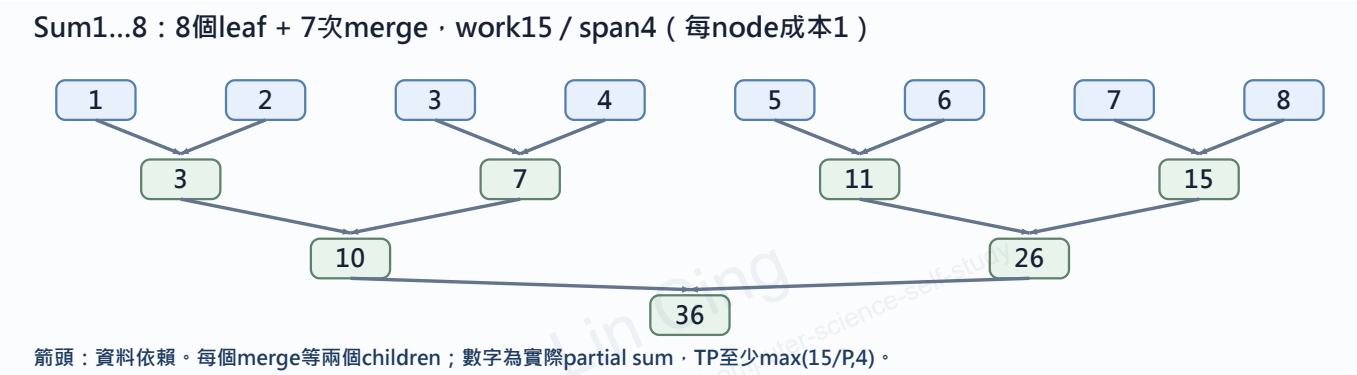


若8core額外啟動/同步開銷總10ms，T8=40ms、speedup2.5。若「80%操作」成本不同就不能直接當80%時間：80個1cycle操作加20個10cycle操作，T1=280；只把前80操作理想加速8倍，T8=10+200=210，speedup=4/3。分母必須維持同一基準。

4.2 Work、span與reduction的依賴限制

固定8個leaf各讀一個值1…8，7個binary merge做加法；每個leaf/merge抽象成本皆1unit。Work=15，最長依賴路徑leaf→pair→quartet→root有4units，span=4。任何P-processor schedule都有TP≥max(15/P,4)，speedup≤15/4；work/span是平均parallelism，不表示超過3.75個processor一律無收益。

4.2 Work、span與reduction的依賴限制 (續)



Pair sums3/7/11/15，quartets10/26，root36，正好7個merges。Tree決定哪些可同時做、哪些必等前驅；scheduler不能把root排在quartets完成之前。若單一worker的partition特別大，總時間至少是最大partition時間，單看平均partition不夠。

4.2 False sharing不等於語言data race

固定cache line64bytes、int4bytes、base對齊64。W1反覆寫a[0]，W2寫a[1]：不同int可沒有同一物件上的data race，但同一cache line可能反覆轉移ownership，降低效能。若各自放在對齊且分隔至少64bytes的區域，能降低此種干擾，仍須實測才能聲稱改善。

把a[0]和a[1]都加mutex可能消除某些應用競爭卻增加序列化；把本來同一counter拆成TLS則改變抽象資料，最後仍需reduce。正確性、共享line效能與結果合併是三個不同問題。

4.3 Multithreading models

- M:1、1:1、M:N與two-level區分user execution到kernel entities的映射；實際可跑數取決於ready carriers與cores。
- Blocking syscall、user-level async等待與blocking compensation是不同模型，不能不附條件說M:N最多某數threads可blocked。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.3 User threads映射到哪些kernel可排程entities

模型	映射	單一blocking syscall的影響	多核能力
Many-to-one	多user threads→1kernel entity	若carrier真阻塞且無async替代，該process其他user threads不能被kernel另排	最多1carrier同時跑
One-to-one	每user thread→1kernel entity	此thread阻塞，其他entities仍可排	受ready數與cores限制
Many-to-many	runtime在多carriers上multiplex	依carrier數與blocking compensation設計	受可跑carriers與cores限制
Two-level	M:N另允許部分thread綁carrier	綁定與unbound各依規則	綁定會限制runtime選擇

User library API不決定唯一映射；不能看到Pthreads就推論M:1或看到很多user threads就推論有同數kernel objects。Runtime若用nonblocking I/O加event loop，可讓很多user tasks邏輯等待而carrier不阻塞，須與真blocking syscall分開。

4.3　100個user threads、10個carriers：固定與補償模型

先固定10個carriers K1…K10，無補償/async；100個user threads皆想發一次blocking I/O。U1使K1阻塞後還有9carriers；10個各自阻塞後，0可跑，剩90個user threads留runtime ready queue，尚無carrier去發新syscalls。此時是10個kernel entities blocked，不是已證所有M:N一律最多10個user-level等待操作。



4.3　100個user threads、10個carriers：固定與補償模型 （續）

另設compensation policy：每個blocking carrier觸發runtime/kernel協作提供替代carrier，最多配置20個。K1…K10阻塞後K11…K20仍可執行，甚至讓更多user threads進blocking state；因此「M:N最多10個同時blocked」不是不附前提的定理。若採async I/O，100個user tasks都可等completion而10carriers未必任何一個阻塞。

Scheduler activations/upcalls是協調kernel與user scheduler的歷史機制：kernel通知阻塞/可用執行資源，runtime在適當context選另一個user thread。它需處理runtime lock、通知context與carrier配置，不會憑空創造CPU算力；two-level綁定也不是讓一個user thread同時在兩個carriers跑。

4.4 Thread libraries

- Pthreads 定義 API 行為，實際實作可用核心 thread 或混合模型。Java thread 常由 runtime 映射到 OS thread。
- 同一個 library call 的 thread cancel、join 與退出語意不同，不能跨 API 任意翻譯。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

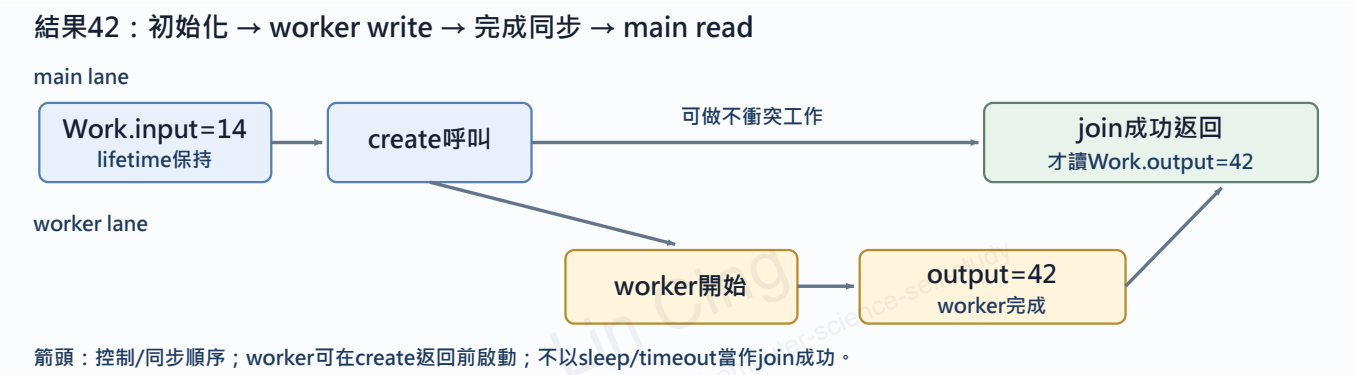
4.4 Create、完成與join建立結果的生命週期

固定main保有Work{input14,output未讀}，worker計output=3×14=42。Main先初始化input再create；Work活到join成功之後，main在join前不讀worker寫的output，也不以未同步state假定worker尚未開始。

階段	main	worker	共享結果規則
create前	input14初始化	尚無worker	物件存在且參數有效
create成功後	可做其他不衝突工作	可已開始/完成、寫output42；先後無固定	parent/child先後無固定保證
worker完成	join可能已等待	不再使用Work	完成與結果publication依library同步
join成功後	讀output42、再結束Work lifetime	已完成	不與worker寫重疊

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.4 Create、完成與join建立結果的生命週期 (續)



Join不是固定sleep；睡10ms沒有保證worker完成。Detached thread不再由join收取，卻仍要由另一協議保證資料物件生命期與完成通知；不能detach後立刻free其參數。

4.4 Pthreads、Windows與Java的相同工作不同介面

Pthreads以pthread_create傳start routine及void*參數，成功pthread_join等joinable target終止並取得返回值；錯誤以函式回傳的error number處理，不能任意只看errno。Join同一target的所有權要明定，多個joiners不能當成任意安全的broadcast；thread退出後不要return自己的stack-local pointer。

Windows案例用CreateThread傳入口/參數，WaitForSingleObject(thread handle)得到成功signaled結果後才讀42；timeout或WAIT_FAILED都不能當完成。Thread handle仍需CloseHandle，關handle不等於殺thread。相應wait/signal操作包含必要memory-order機制，不能只靠volatile代替同步。

Java8中Thread.start讓新thread執行run，直接呼叫run只是當前thread普通method call。啟動前的actions happens-before新thread；worker全部actions happens-before另一thread成功join返回，故例中可在join後讀42。InterruptedException不是完成證明，不能吞掉後直接讀結果。語言/API契約與host kernel映射是兩層問題。

4.5 Implicit threading

- thread pool 限制同時工作量並重用執行者；fork-join 遞迴拆解工作；OpenMP 以 compiler 指引平行化迴圈或區段。
- Grand Central Dispatch 等方法讓 runtime 管理工作排程，但呼叫者仍需處理共享狀態與依賴。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.5 Thread pool與fork-join減少建立成本，也有等待風險

固定8jobs各CPU1unit，4workers、FIFO且無overhead；第一批J1–J4在[0,1)，第二批J5–J8在[1,2)，makespan2。此理想模型的worker建立不是每job重做，但任意大量排入仍需bounded queue/backpressure與取消/錯誤處理。Worker數與queue容量是兩個不同資源上限。

Fork-join將問題分成獨立subtasks再合併；4.2的8輸入sum中葉值1…8、partials3/7/11/15、10/26、36。只有每個依賴都完成才可merge。Granularity過小可使task/queue/synchronization成本大於useful work，不能只看task數增加。

若2worker pool同時跑A/B，兩者各submit child A1/B1後blocking wait，而queue中的A1/B1都等空worker，會形成pool starvation deadlock。Work-stealing/help-join runtime可能讓等待者協助執行ready child，但不是所有pool/Future自動具此能力；無環task DAG也可能因不當blocking策略卡住。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.5 OpenMP reduction與GCD queue的機制

本書OpenMP模型以parallel for切iteration；獨立c[i]=a[i]+b[i]可分不同i，但sum+=a[i]寫同sum需要reduction/同步。以4workers sum1…8，每人兩項得到3/7/11/15，combine10+26=36。Reduction使用private partials再按規則合併，不是讓所有threads對plain sum任意同寫。Floating-point加法不具精確結合律，reassociation可能改最後bits。

本書GCD以dispatch queues提交blocks，runtime管理worker pool。Serial queue保證該queue上的blocks一次一個完成；concurrent queue允許重疊，enqueue FIFO不代表所有blocks依FIFO完成。若正在serial Q執行的block同步提交另一block到同Q並等它，後者又須等前者離場，形成self-wait；不同queue之間共享物件也沒有自動互斥。

Implicit threading把排程/建立交給runtime，應用仍負責正確劃分依賴、shared/private資料、lifetime與不安全blocking。加一個parallel directive不能把有loop-carried dependency的程式自動變成正確獨立工作。

4.6 Threading issues

- fork 在 multi-thread process 中要決定 child 複製全部 threads 或只保留呼叫者。signal 也可能由哪個 thread 處理而造成問題。
- thread cancellation 分 deferred 與 asynchronous。deferred cancellation 讓 thread 在取消點清理資源，較能避免半完成狀態。

4.6 Multithread fork與signal delivery

POSIX-style multithread process有T1/T2，T2持有mutex M時T1 fork。Child只保留caller T1，記憶體中的M卻可能仍顯示由不存在的T2持有；child若在exec前呼叫需要M的普通library函式就可能無法前進。Child在exec前只可用相應async-signal-safe操作；atfork handlers需配合整體鎖/函式庫設計，不能無條件修好未知第三方鎖。

Signal disposition一般屬process共享；signal mask屬各thread。Synchronous fault信號指向造成事件的thread；process-directed signal由符合mask條件的thread按實作規則接收，不能假定所有threads都跑一次handler。使用專門sigwait thread時應協調其他threads遮罩，不把signal與condition variable當同一物件。

Exec成功後替換process image，原其他threads不繼續舊image；普通建立thread則共享現有image。這些不同操作決定resources/locks如何繼承，不能只數API名稱。

4.6 Deferred cancellation要在可清理狀態生效

pthread_cancel成功只表示請求已排入，target可能disabled或尚未到cancellation point，不能當作已終止。Deferred模式在指定points處理；asynchronous可能在任意不安全片段生效，會留下半更新資料或仍持有的locks。

固定worker持M並擁有buffer B，在穩定predicate上pthread_cond_wait。事前以cancellation-disabled建立資源與cleanup handlers，依序push unlock-M、push free-B；恢復deferred cancellation後，若在cond wait取消，依API先重新取得M，再以LIFO執行free-B、unlock-M。Main成功join後才回收joinable thread相關狀態。不能在cleanup註冊前就開放任意取消。

4.6　取消前後的資源表與不變量

狀態	M	B	free-B次數	unlock-M次數
資源與handlers已建好	worker持有	live	0	0
cond wait期間	原子釋M等待	live	0	0
cancellation被處理	依cond-wait契約重取得M	live	0	0
第一個cleanup	worker仍持M	freed	1	0
第二個cleanup	unlocked	freed	1	1

正常路徑也要pop/執行對應清理一次；不能正常free一次又在取消cleanup再free。清理只釋放資源仍不足以恢復任意半更新資料：在不變量暫破期間應禁止取消/不做cancellation point，或提供真正rollback。取消協議要包含normal/error/timeout路徑與誰最後擁有每個物件。

4.7　作業系統例子

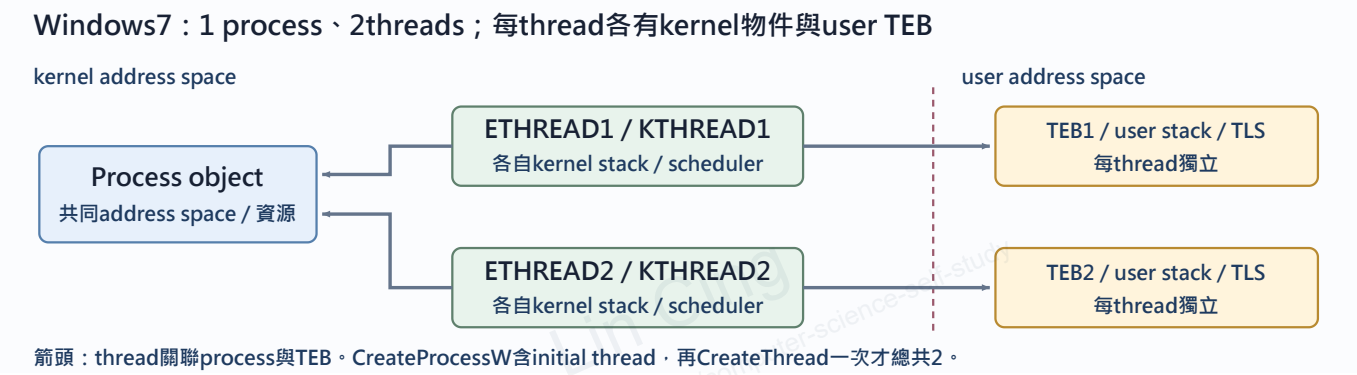
- Windows 7 案例以 process object 容納 address space 與資源，thread 以 ETHREAD、KTHREAD、TEB 表示；CreateProcessW 建立 process，CreateThread 建立可被核心排程的 thread。
- Linux 3.2 以 task_struct 表示每個可排程 task。fork() 經 copy_process() 與 dup_task_struct() 建立獨立 task，未帶 CLONE_VM 時另有獨立 mm 與頁表；clone() 依 flags 共享或複製 fs、files、sighand 與 memory context。
- 兩案例的共同點是排程單位與 address-space ownership 必須分開觀察。Linux thread-like task 可共享 mm；Windows thread 一定有指回 process 的關係。版本快照不能外推成今日核心的保證。

4.7　Windows 7 與 Linux 3.2 的 thread 物件對照

本頁固定 OSC 9e 的版本快照。Windows 7 案例以 process 保存 address space 與資源，thread 由 ETHREAD、KTHREAD 與 TEB 表示；此例CreateProcessW先建立process與initial thread，再呼叫CreateThread一次得到第二thread；兩者皆為one-to-one，不能忽略CreateProcessW已建立的initial thread。session 0 的 system threads 存在，但它們不屬於一般 interactive process 的兩個 thread 範例。

Linux 3.2 以 `task_struct` 表示每一個 schedulable task。`fork()` 經 `copy_process()` 呼叫 `dup_task_struct()` 建立新 task 與 kernel stack；未帶 `CLONE_VM` 時，`copy_mm()` 走 `dup_mm()`，因此 child 另有 mm 與頁表。`clone()` 則依 flags 決定 `fs_struct`、`files_struct`、`sighand_struct` 與 mm 的分享。

4.7　Windows 7 與 Linux 3.2 的 thread 物件對照 (續)



4.7 Windows 7 與 Linux 3.2 的 thread 物件對照 (續)

比較項	Windows 7	Linux 3.2
建立 process	`CreateProcessW`	`fork()`
建立 thread-like execution	`CreateThread`	`clone(CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD)`
核心執行物件	Process object + Thread object	每個執行流一個 `task_struct`
thread 私有狀態	registers、user/kernel stacks、TEB、TLS	per-task kernel stack、scheduler state、registers/stack context
shared process state	address space、process-level resources	mm、files、fs、signal handlers 依 clone flags 共享
scheduling unit	kernel thread	task

4.7 Windows 7 與 Linux 3.2 的 thread 物件對照 (續)

固定工作負載是「一個 process 內兩個執行流」。Windows 7 的計數為 1 process object、2 ETHREAD、2 KTHREAD、2 TEB、1 address space、2 scheduling entities。Linux 的一般 fork 對照為 2 `task_struct`、2 mm、2 page tables；再用上述 thread-like clone 對照，計數變成 2 `task_struct`、1 mm、1 file table、1 signal-handler table、2 kernel stacks。

4.7　建立呼叫後的狀態與版本反例

Windows 的第一個 thread 建立後，ETHREAD 可追溯到 process，KTHREAD 保存 scheduling/synchronization state，TEB 位於 user space。第二個 thread 擁有另一組 stacks、registers 與 TEB，但兩個 thread 的 pointer 指向同一 process/address space。Thread ID 或 object state 改變時，另一個 thread 的 identity 仍獨立。

Linux 的 `fork()` child 與 parent 都可在同一段 user code 返回，但不能把「共享 COW page」說成共享 task object。child 的 PID、state、kernel stack、mm 與 page table 已建立；共享的是特定 physical frames 的 COW mapping。若 child 執行 `exec()`，它保留 task identity，成功換成新 program image；exec在尚未破壞舊image前失敗通常可回錯誤；v3.2越過flush_old_exec之後某些失敗會終止，不能無條件保證舊image可繼續。

此處thread-group clone另含CLONE_THREAD，使TID不同而TGID相同；CLONE_THREAD要求CLONE_SIGHAND，後者要求CLONE_VM。只列VM/FS/FILES/SIGHAND並帶SIGCHLD仍可能共享資源卻不屬同一thread group，不能等同完整pthread建立。兩task共享mm與backing不等於未同步的heap寫入自動正確；publication仍需相應同步。一般fork的private page則在第一次寫入時分離。

4.7　建立呼叫後的狀態與版本反例 （續）



上頁的 2/2/2 與 2 task/1 mm 是依固定案例結構建立的教材模型，不是對任何今日核心的 ABI 保證。

4.8　以結果、完成契約與依賴評估threads

同一個sum1…8結果36可由單thread、4workers各算一段，或fork-join reduction完成；差異在工作排程、共享物件與同步成本，不在「有threads所以結果自動正確」。Join後讀partials、按明定operation合併與維持物件lifetime，才使結果可用。

Fixed-size Amdahl以基準時間比例算20+80/P；work/span以DAG相依給下界；carriers限制kernel能排的執行實體。三者解釋不同瓶頸，不能互換成「threads等於cores就最佳」或「超過平均parallelism都沒有收益」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.8　實作改變哪一項，便重新檢查哪個前提

從1:1改M:N要重新看blocking/compensation；從per-thread scratch改global要重新看衝突；從join改detach要另外證明lifetime/完成；從單thread fork改multithread fork要處理遺留locks；從正常return加入cancel要確保資源與抽象資料都恢復。

比較真系統時，須固定輸入與正確結果、cores/carriers、記憶體配置與時間分母，再量測throughput與延遲分布。平均吞吐可能掩蓋lock contention、pool耗盡或取消清理造成的長尾，因此也要保留失敗次數與等待位置。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料

- *Operating System Concepts*, 9th Edition, Chapter4，印刷頁163–191；Fig4.6/4.7為thread映射，Fig4.14/4.15為Windows結構及Linux flags；歷史案例固定Windows7、Linux3.2與本書GCD/OpenMP模型。
- Linuxv3.2 kernel/fork.c：THREAD/SIGHAND/VM依賴及獨立task/共享contexts。
- pthread_join(3)、pthread_cancel(3)、pthread_cleanup_push(3)：joinable完成、取消請求、LIFO清理與錯誤契約。
- POSIX pthread_cond_wait：atomic unlock/wait、重檢predicate、deferred cancellation前重新取得mutex。
- Oracle Java8 JLS17.4.5：start/join的happens-before。
- Microsoft Synchronization and Multiprocessor Issues：wait與其他同步原語的ordering；kernel版本案例與公開API契約分開。

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study